Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Лабораторна робота № 3

з дисципліни «Протоколи та алгоритми електронного голосування»

Виконав: студент групи ІП-34 Туболев М. Р.

Перевірив: Асистент кафедри, PhD Нестерук А. О.

2026

1. Тема, мета та завдання роботи

Тема: розподіл повноважень у протоколах Е-голосування. Частина 1.

Мета роботи: програмно реалізувати всі кроки протоколу електронного голосування з розподілом повноважень між бюро реєстрації (БР) та виборчою комісією (ВК) для двох кандидатів і п’яти виборців, використовуючи алгоритм RSA як для шифрування повідомлень, так і для електронного цифрового підпису (ЕЦП); дослідити, як розділення даних між БР і ВК відокремлює особу виборця від його голосу та які можливості для шахрайства при цьому залишаються.

Завдання роботи:

- реалізувати БР і ВК у вигляді окремих програмних модулів, кожен з яких зберігає лише ті дані, які відповідна сторона отримує згідно з протоколом, а обмін між ними відбувається без прямого доступу до внутрішніх даних один одного;
- реалізувати формування БР списку виборців, перевірку особи виборця, його права брати участь у голосуванні та факту попередньої реєстрації;
- реалізувати самостійну генерацію кожним виборцем пари ключів RSA для ЕЦП, генерацію БР випадкового унікального RN для кожного зареєстрованого виборця та збереження зв’язку «виборець — RN — відкритий ключ»;
- реалізувати передачу від БР до ВК списку дійсних RN з відкритими ключами без персональних даних, генерацію ВК власної пари ключів RSA та публікацію відкритого ключа;
- реалізувати самостійну генерацію виборцем випадкового ID, формування повідомлення (ID; RN; бюлетень), його підписання закритим ключем виборця та шифрування відкритим ключем ВК;
- реалізувати розшифрування повідомлення ВК, перевірки наявності RN у списку, його попереднього використання, ЕЦП за відповідним відкритим ключем і унікальності ID, прийняття або відхилення бюлетеня та позначення RN як використаного;
- реалізувати підрахунок голосів, публікацію результатів зі списком ID і підписаних бюлетенів, а також перевірку виборцем власного голосу

з повідомленням про невідповідність;

– для кожної перевірки показати як позитивний, так і негативний сценарій.

2. Короткий опис програмної реалізації

Протокол реалізовано мовою TypeScript і виконується середовищем Node.js без етапу збірки. Криптографічні бібліотеки не використовувалися: модульна арифметика, генерація простих чисел, розширений алгоритм Евкліда, блокове шифрування RSA, хеш квадратичної згортки та ЕЦП написані вручну (ці модулі перенесено з лабораторної роботи № 1). Ключі всіх учасників мають довжину 512 біт.

Кожна сторона протоколу — окремий модуль і окремий клас. Внутрішні дані класів RegistrationBureau та ElectionCommission оголошено як приватні поля мови JavaScript (#registry, #rns, #ballots тощо), тому до них неможливо звернутися ззовні навіть у демонстраційному сценарії. БР і ВК не мають посилань одна на одну: список RN передається як серіалізований текст (рядок JSON) разом з ЕЦП БР, а ВК самостійно розбирає його і будує власний реєстр. До ВК від виборця надходить лише шифротекст — без імені відправника.

Модуль	Призначення
src/bigint-math.ts	модульне піднесення до степеня, розширений алгоритм Евкліда, обернене за модулем, тест простоти Міллера — Рабіна, генерація простих чисел і випадкових чисел у діапазоні
src/rsa.ts	генерація пари ключів, блокове шифрування та розшифрування, хеш квадратичної згортки, формування і перевірка ЕЦП
src/types.ts	структури звернення до БР, запису реєстру, списку RN, повідомлення (ID; RN; бюлетень), опублікованого бюлетеня, причини відмов, канонічна серіалізація
src/bureau.ts	бюро реєстрації: реєстр громадян, перевірки виборця, видача RN, реєстр «виборець — RN — ключ», підписаний список RN для ВК, публікація списку виборців
src/commission.ts	виборча комісія: прийняття списку RN, власні ключі, розшифрування, перевірки, позначення RN, підрахунок і публікація результатів
src/voter.ts	виборець: власні ключі ЕЦП, звернення до БР, генерація ID, підписання та шифрування повідомлення, перевірка власного голосу
src/main.ts	демонстраційний сценарій: позитивні та негативні випадки всіх перевірок, аналіз можливостей шахрайства
tests/	25 модульних тестів (вбудований тест-ранер node:test)

Таблиця 1 — Структура програмної реалізації

Протокол відповідає описаному в теоретичних відомостях до лабораторної роботи та протоколу з двома ВК з лекції 13: БР відповідає комісії реєстрації (КР), RN — реєстраційному номеру Р№, повідомлення (ID; RN; бюлетень) — повідомленню M(Id; Р№; BL). Відповідно до зауваження лекції 13 про те, що всі повідомлення мають бути зашифровані та підписані, реалізація додатково підписує список RN ключем БР, а ВК перевіряє цей підпис перед прийняттям списку. Також БР перемішує записи списку, а ВК публікує бюлетені впорядкованими за ID, щоб порядок записів не відтворював порядок реєстрації чи голосування.

2.1. Генерація ключів RSA та ЕЦП

Кожен учасник (виборець, БР, ВК) генерує власну пару ключів: обираються два різні прості числа p і q, обчислюються $n = p \cdot q$ та $\varphi(n) = (p-1)(q-1)$, відкритий показник $e = 65537$ (взаємно простий з $\varphi(n)$) і закритий показник $d = e^{-1} \bmod \varphi(n)$ за розширеним алгоритмом Евкліда. Перед підписанням повідомлення хешується функцією квадратичної згортки $H_i =$

$(H_{i-1} + M_i)^2 \bmod n$; ЕЦП = H^d mod n. Перевіряючий порівнює хеш отриманого тексту з H_c = ЕЦП^e mod n.

```
/**
 * Генерація пари ключів RSA.
 * p, q — різні прості; n = p*q; φ(n) = (p-1)(q-1);
 * e — непарне, 1 < e < φ(n), НСД(e, φ(n)) = 1; d = e⁻¹ mod φ(n).
 */
export function generateKeyPair(bits = 512): KeyPair {
  if (bits < 64) throw new RangeError("Для блокового шифрування потрібно
щонайменше 64 біти");

  const halfBits = Math.floor(bits / 2);
  let p = generatePrime(halfBits);
  let q = generatePrime(bits - halfBits);
  while (p === q) q = generatePrime(bits - halfBits);

  const n = p * q;
  const phi = (p - 1n) * (q - 1n);

  const e = gcd(DEFAULT_PUBLIC_EXPONENT, phi) === 1n
    ? DEFAULT_PUBLIC_EXPONENT
    : findCoprimeExponent(phi);
  const d = modInverse(e, phi); // розширений алгоритм Евкліда

  return { publicKey: { e, n }, privateKey: { d, n }, parameters: { p, q,
phi } };
}
```

Лістинг 1 — Генерація пари ключів (src/rsa.ts)

```
/**
 * Хеш квадратичної згортки: Hᵢ = (Hᵢ₋₁ + Mᵢ)² mod n, H₀ = 0,
 * де Mᵢ — числове представлення символу повідомлення.
 */
export function hashMessage(message: string, n: bigint): bigint {
  const bytes = new TextEncoder().encode(message);
  let h = 0n;
  for (const byte of bytes) h = ((h + BigInt(byte)) ** 2n) % n;
  return h;
}

/** Формування ЕЦП закритим ключем: ЕЦП = H^d mod n. */
export function sign(message: string, key: PrivateKey): bigint {
  return modPow(hashMessage(message, key.n), key.d, key.n);
}

/** Перевірка ЕЦП: порівнюємо H і H_c = ЕЦП^e mod n. */
export function verify(message: string, signature: bigint, key: PublicKey):
boolean {
  const expected = hashMessage(message, key.n);
  const recovered = modPow(signature, key.e, key.n);
  return expected === recovered;
}
```

Лістинг 2 — Хеш, формування та перевірка ЕЦП (src/rsa.ts)

2.2. Бюро реєстрації: перевірки виборця та видача RN

БР створюється з реєстром громадян (ПІБ, номер паспорта, дата народження) та датою виборів. Виборець звертається до БР з ПІБ, паспортом і власним відкритим ключем ЕЦП. БР послідовно перевіряє: наявність паспорта в реєстрі громадян; відповідність ПІБ паспортним даним (перевірка особи); право голосу — особі на день виборів має виповнитися 18 років; відсутність попередньої реєстрації за цим паспортом. Повторна реєстрація заборонена незалежно від того, з яким ключем звертається виборець, тому один виборець може отримати лише один RN. Після успішних перевірок БР генерує випадковий RN з діапазону 100000–999999 (значно більшого за кількість виборців, як вимагає лекція 13), гарантує його унікальність і зберігає у своєму реєстрі зв’язок «виборець — RN — відкритий ключ ЕЦП».

```
/** Кроки 1–2: перевірка виборця та видача йому RN. */
register(request: RegistrationRequest): RegistrationResult {
  const citizen = this.#roll.get(request.passport);
  if (!citizen) {
    this.#note("inRoll", false, `паспорта ${request.passport} немає в
реєстрі громадян`);
    return refuse("NOT_IN_ROLL");
  }
  this.#note("inRoll", true, `паспорт ${request.passport} є в реєстрі
громадян`);

  const identityMatches = citizen.name === request.name;
  this.#note(
    "identity",
    identityMatches,
    identityMatches
      ? `паспорт належить «${citizen.name}»`
      : `за цим паспортом у реєстрі — «${citizen.name}»`,
  );
  if (!identityMatches) return refuse("IDENTITY_MISMATCH");

  const age = ageOn(citizen.birthDate, this.#electionDate);
  const hasRight = age >= ADULT_AGE;
  this.#note("votingRight", hasRight, `на день виборів
${this.#electionDate} виповниться ${age} р.`);
  if (!hasRight) return refuse("NO_VOTING_RIGHT");

  if (this.#registry.has(request.passport)) {
    this.#note("notRegistered", false, `«${citizen.name}» уже має RN`);
    return refuse("ALREADY_REGISTERED");
  }
  this.#note("notRegistered", true, `«${citizen.name}» раніше не
реєструвався`);

  const rn = this.#issueRn();
  this.#registry.set(request.passport, { ...request, rn });
  return { status: "registered", rn };
}
```

```
#issueRn(): string {
  while (true) {
    const rn = randomBigInt(RN_MIN, RN_MAX).toString();
    if (!this.#issuedRns.has(rn)) {
      this.#issuedRns.add(rn);
      return rn;
    }
  }
}
```

Лістинг 3 — Реєстрація виборця та генерація унікального RN (src/bureau.ts)

2.3. Передача списку RN від БР до ВК

Після реєстрації БР формує список дійсних RN і відповідних їм відкритих ключів — без ПІБ і паспортів — у вигляді канонічного тексту, підписує його своїм закритим ключем і передає до ВК. ВК перевіряє ЕЦП під списком відкритим ключем БР і лише після цього будує власний реєстр, у якому для кожного RN зберігаються відкритий ключ ЕЦП і позначка використання. Підроблений або змінений після підписання список відхиляється. Окремо БР публікує список зареєстрованих виборців без RN для контролю кількості бюлетенів.

```
/** Крок 3: список дійсних RN і відкритих ключів без персональних даних,
підписаний БР. */
exportRnList(): RnListMessage {
  const payload = serializeRnList(
    shuffle([...this.#registry.values()]).map(({ rn, publicKey }) => ({ rn,
publicKey })),
  );
  return { payload, signature: sign(payload, this.#keys.privateKey) };
}

/** Крок 3: прийняття списку RN від БР — лише якщо його підписала саме БР.
*/
importRnList(message: RnListMessage, bureauKey: PublicKey): RnListImport {
  const authentic = verify(message.payload, message.signature, bureauKey);
  this.#note(
    "rnListSignature",
    authentic,
    authentic ? "ЕЦП БР під списком RN справжній" : "ЕЦП під списком RN не
належить БР",
  );
  if (!authentic) return { ok: false, error: "підпис не належить БР" };

  const records = parseRnList(message.payload);
  for (const { rn, publicKey } of records) this.#rns.set(rn, { publicKey,
used: false });
  return { ok: true, count: records.length };
}
```

Лістинг 4 — Експорт списку RN у БР та його прийняття у ВК (src/bureau.ts, src/commission.ts)

2.4. Дії виборця: ID, повідомлення, ЕЦП і шифрування

Виборець самостійно генерує випадковий ID із 16 десяткових цифр (імовірність збігу у двох виборців мізерна), формує повідомлення (ID; RN; бюлетень) у канонічному вигляді, підписує його власним закритим ключем і шифрує пару «повідомлення + ЕЦП» відкритим ключем ВК. Імені виборця в повідомленні немає. Виборець запам’ятовує свій ID і зроблений вибір, щоб після публікації перевірити врахування власного голосу.

```
/** Крок 4: сформувати повідомлення (ID; RN; бюлетень) і підписати його
закритим ключем. */
prepare(candidateId: string, id = generateId()): SignedMessage {
  const text = serializeVoteMessage({ id, rn: this.rn, candidateId });
  return { text, signature: sign(text, this.keys.privateKey) };
}

/** Повний крок виборця: сформувати → підписати → зашифрувати ключем ВК;
виборець запам’ятовує свій ID і вибір. */
vote(candidateId: string, commissionKey: PublicKey): { signed:
SignedMessage; envelope: Envelope } {
  const id = generateId();
  const signed = this.prepare(candidateId, id);
  this.#cast = { id, candidateId, text: signed.text };
  return { signed, envelope: sealFor(signed, commissionKey) };
}

/**
 * Шифрування підписаного повідомлення разом з ЕЦП відкритим ключем ВК.
 * Винесено окремо, бо відкритий ключ ВК доступний будь-кому —
 * цим користуються негативні сценарії з підміною повідомлення.
 */
export function sealFor(signed: SignedMessage, commissionKey: PublicKey):
Envelope {
  const payload = JSON.stringify({ text: signed.text, signature:
signed.signature.toString(16) });
  return { cipher: encrypt(payload, commissionKey) };
}
```

Лістинг 5 — Формування, підписання та шифрування повідомлення (src/voter.ts)

2.5. Перевірки виборчої комісії

ВК розшифровує отримане повідомлення власним закритим ключем і послідовно перевіряє: наявність RN у списку від БР; що RN ще не використаний; ЕЦП за відкритим ключем, закріпленим саме за цим RN; унікальність ID серед прийнятих бюлетенів; коректність заповнення бюлетеня. Якщо хоча б одна перевірка не пройдена, бюлетень відхиляється, а

RN залишається вільним — виборець може надіслати виправлений бюлетень. Після успішних перевірок ВК позначає RN як використаний і зберігає підписаний бюлетень разом з ID.

```
/** Крок 5: розшифрування повідомлення та всі передбачені протоколом
перевірки. */
receive(envelope: Envelope): ProcessingResult {
  const opened = this.#open(envelope);
  if (!opened.ok) {
    this.#note("decrypt", false, opened.error);
    return this.#reject("UNDECRYPTABLE");
  }
  const { message, text, signature } = opened;
  this.#note("decrypt", true, `ID = ${message.id}, RN = ${message.rn},
бюлетень: «${message.candidateId}»`);

  const entry = this.#rns.get(message.rn);
  if (!entry) {
    this.#note("rnInList", false, `RN ${message.rn} немає у списку від
БР`);
    return this.#reject("UNKNOWN_RN");
  }
  this.#note("rnInList", true, `RN ${message.rn} є у списку від БР`);

  if (entry.used) {
    this.#note("rnNotUsed", false, `RN ${message.rn} уже позначений як
використаний`);
    return this.#reject("RN_USED");
  }
  this.#note("rnNotUsed", true, `RN ${message.rn} ще не використовувався`);

  const signatureValid = verify(text, signature, entry.publicKey);
  this.#note(
    "signature",
    signatureValid,
    signatureValid ? "H = H_c, ЕЦП відповідає ключу цього RN" : "H ≠ H_c,
ЕЦП не відповідає ключу цього RN",
  );
  if (!signatureValid) return this.#reject("BAD_SIGNATURE");

  if (this.#ballots.has(message.id)) {
    this.#note("idUnique", false, `ID ${message.id} уже є серед прийнятих
бюлетенів`);
    return this.#reject("DUPLICATE_ID");
  }
  this.#note("idUnique", true, `ID ${message.id} унікальний`);

  const candidate = this.candidates.find((item) => item.id ===
message.candidateId);
  if (!candidate) {
    this.#note("ballotValid", false, `кандидата «${message.candidateId}»
немає у списку`);
    return this.#reject("INVALID_BALLOT");
  }
  this.#note("ballotValid", true, `голос за кандидата
«${candidate.name}»`);

  entry.used = true;
```

```
  this.#ballots.set(message.id, { id: message.id, candidateId:
candidate.id, message: text, signature });
  return { status: "accepted", id: message.id, candidateId: candidate.id };
}
```

Лістинг 6 — Обробка повідомлення виборчою комісією (src/commission.ts)

2.6. Публікація результатів і перевірка виборцем свого голосу

Після завершення голосування ВК підраховує голоси й публікує результати разом зі списком ID і відповідних підписаних бюлетенів. Кожен виборець шукає у списку свій ID і порівнює опублікований бюлетень (і вибір, і підписаний текст) з тим, що надсилав. Якщо ID відсутній або бюлетень не збігається зі зробленим вибором, виводиться повідомлення про порушення.

```
/** Крок 6: підрахунок голосів і публікація результатів разом зі списком
(ID; підписаний бюлетень). */
publishResults(): ElectionResults {
  const ballots = [...this.#ballots.values()].sort((a, b) =>
a.id.localeCompare(b.id));
  const tally = new Map(this.candidates.map((candidate) => [candidate.id,
0]));
  for (const ballot of ballots) tally.set(ballot.candidateId,
(tally.get(ballot.candidateId) ?? 0) + 1);

  const max = Math.max(0, ...tally.values());
  const leaders = max === 0 ? [] : this.candidates.filter((candidate) =>
tally.get(candidate.id) === max);

  return {
    tally,
    ballots,
    rejected: this.#rejected,
    registered: this.#rns.size,
    leaders,
    isTie: leaders.length > 1,
  };
}
```

Лістинг 7 — Публікація результатів (src/commission.ts)

```
/** Після публікації: знайти власний ID і переконатися, що бюлетень
врахований без змін. */
verifyPublication(ballots: readonly PublishedBallot[]): VerificationResult
{
  if (!this.#cast) throw new Error(`«${this.name}» ще не голосував`);
  const { id, candidateId, text } = this.#cast;

  const published = ballots.find((ballot) => ballot.id === id);
  if (!published) return { status: "missing", id };
  if (published.candidateId !== candidateId || published.message !== text)
{
    return { status: "mismatch", id, expected: candidateId, published:
published.candidateId };
  }
```

```
  return { status: "confirmed", id };
}
```

Лістинг 8 — Перевірка виборцем власного голосу (src/voter.ts)

3. Результати виконання протоколу та передбачених перевірок

Демонстраційний сценарій запускається командою npm start. У ньому беруть участь два кандидати та п’ять виборців; для кожної перевірки БР і ВК передбачено як позитивний, так і негативний випадок. Зведення сценаріїв наведено в таблиці 2.

Сценарій	**Що перевіряється**	**Результат**
Реєстрація 5 виборців	усі перевірки БР	видано 5 різних RN
Р1. Повторне звернення по RN (з новим ключем)	відсутність попередньої реєстрації	відмова: ALREADY_REGISTERED
Р2. Особа поза реєстром громадян	наявність у реєстрі	відмова: NOT_IN_ROLL
Р3. Чужий паспорт	перевірка особи	відмова: IDENTITY_MISMATCH
Р4. Неповнолітня особа	право голосу	відмова: NO_VOTING_RIGHT
П1. Підроблений / змінений список RN	ЕЦП БР під списком	список відхилено
П2. Справжній список RN	ЕЦП БР під списком	список прийнято, 5 RN у ВК
Коректні бюлетені (5 виборців)	усі перевірки ВК	прийнято, RN позначено
Г1. Вгаданий RN	наявність RN у списку	відхилено: UNKNOWN_RN
Г2. Повторне голосування	попереднє використання RN	відхилено: RN_USED
Г3. Повторний ID	унікальність ID	відхилено: DUPLICATE_ID
Г4. Викрадений RN, чужий ключ	ЕЦП за ключем RN	відхилено: BAD_SIGNATURE
Г5. Підміна після підписання	цілісність: $H \neq H_c$	відхилено: BAD_SIGNATURE
Г6. Неіснуючий кандидат	коректність бюлетеня	відхилено: INVALID_BALLOT
Г7. Зашифровано не ключем ВК	розшифрування	відхилено: UNDECRYPTABLE
Чесна публікація	перевірка виборцем	усі 5 голосів підтверджено
Підміна / вилучення бюлетеня ВК	перевірка виборцем	виборці №2 і №5 бачать повідомлення про порушення

Таблиця 2 — Перевірені сценарії роботи протоколу

На рисунку 1 показано кроки 1–2: реєстр громадян, з яким працює БР, пари ключів ЕЦП, які виборці згенерували самостійно, проходження чотирьох перевірок БР, видачу RN та реєстр БР зі зв’язком «виборець — RN — відкритий ключ».



Рисунок 1 — Кроки 1–2: реєстрація виборців у БР та видача RN

Рисунок 2 демонструє негативні сценарії реєстрації. Виборець, який уже отримав RN, не може отримати другий навіть з новою парою ключів; особа без паспорта в реєстрі, особа з чужим паспортом і неповнолітня особа

отримують відмову на відповідній перевірці. Для порівняння показано виборця, якому 18 років виповнилося якраз до дня виборів.

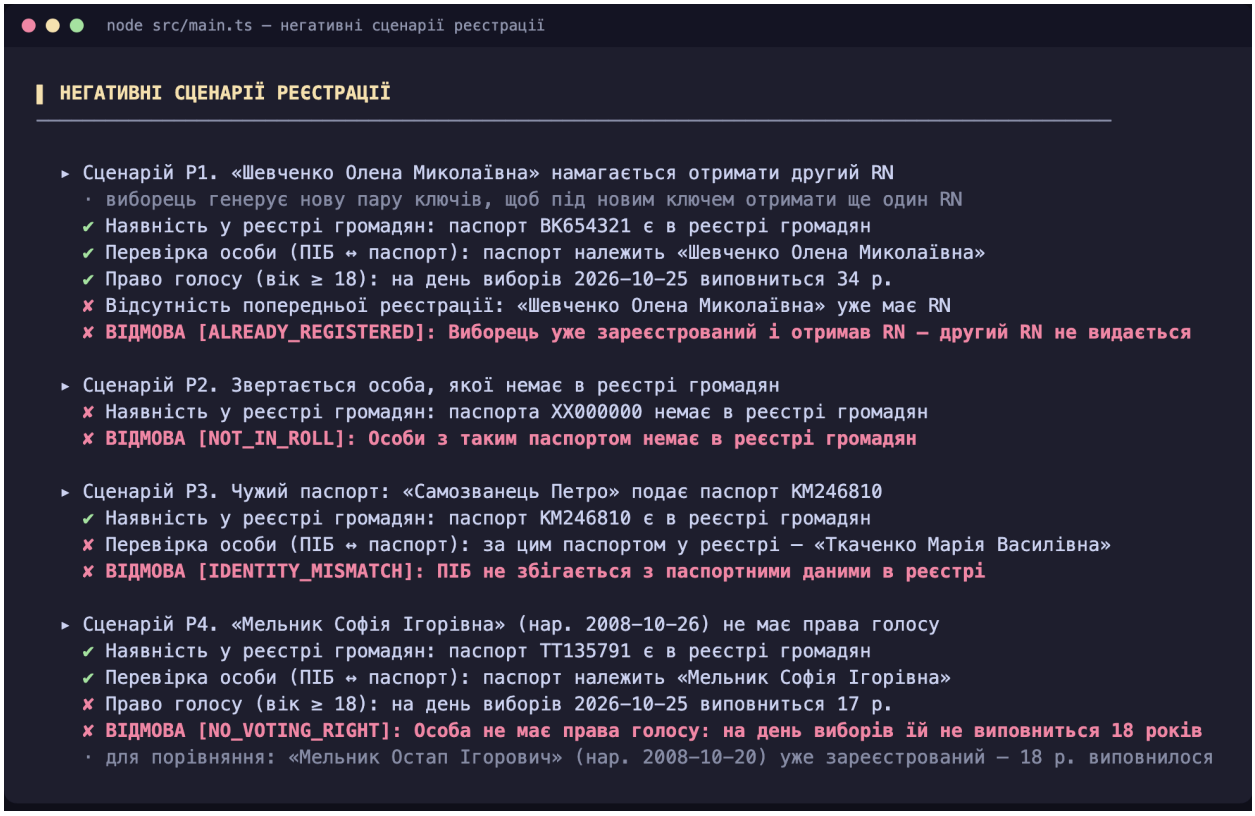


Рисунок 2 — Негативні сценарії реєстрації

На рисунку 3 показано крок 3: БР формує підписаний список RN без персональних даних. Спершу ВК відхиляє підроблений список і справжній список з дописаним RN, а потім приймає справжній. У реєстрі ВК присутні лише RN, відкриті ключі та позначка використання; також показано опублікований відкритий ключ ВК і список зареєстрованих виборців без RN.

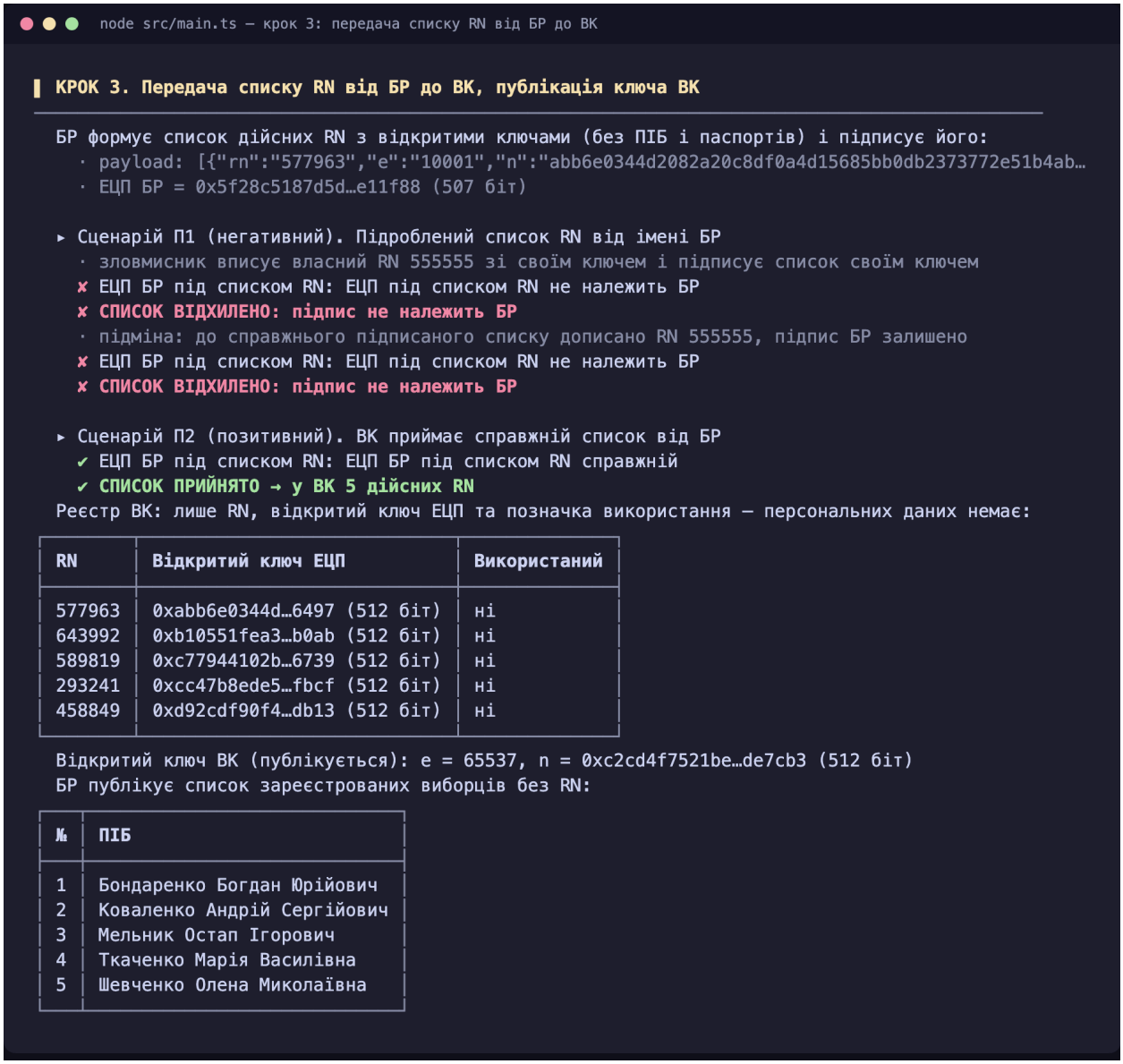


Рисунок 3 — Крок 3: передача списку RN від БР до ВК

На рисунку 4 наведено позитивний сценарій голосування: повідомлення (ID; RN; бюлетень), його хеш, ЕЦП, блоки шифротексту та послідовне проходження всіх перевірок ВК.

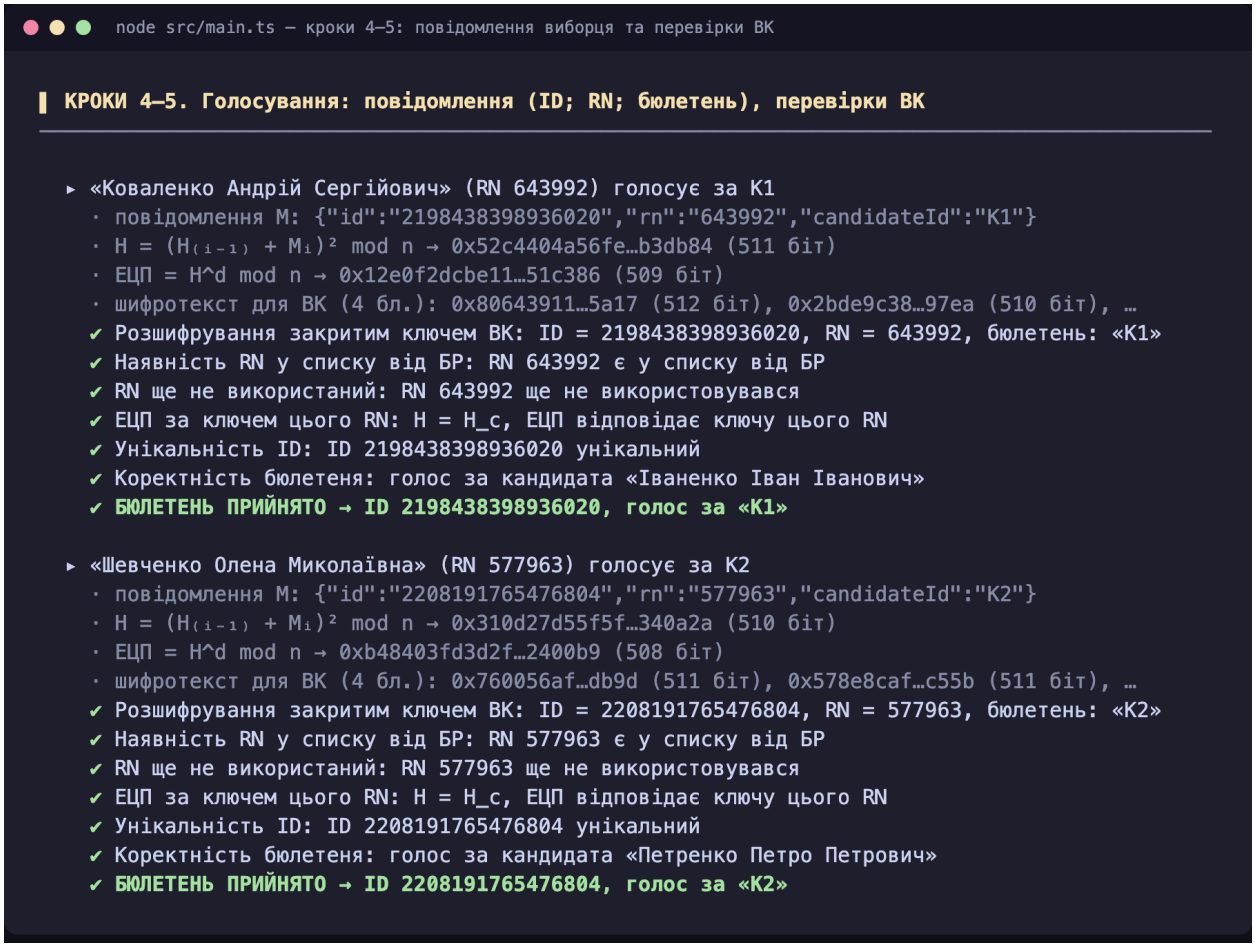


Рисунок 4 — Кроки 4–5: формування повідомлення та прийняття бюлетеня

Рисунки 5 і 6 демонструють негативні сценарії голосування: бюлетень з вгаданим RN, повторне голосування з тим самим RN, повтор ID, підпис чужим ключем під викраденим RN, підміну бюлетеня після підписання, зіпсований бюлетень і повідомлення, зашифроване не ключем ВК. Кожну спробу зупиняє саме та перевірка, для якої її сформовано.

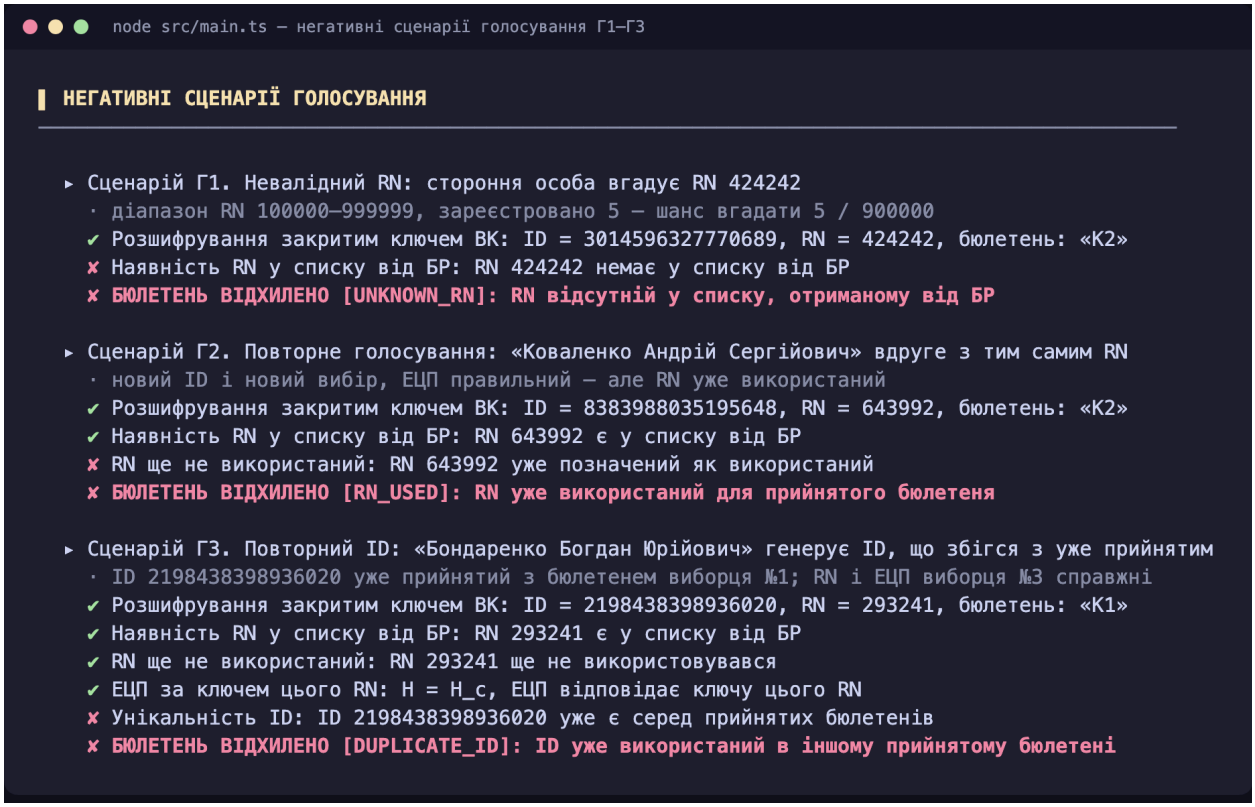


Рисунок 5 — Негативні сценарії голосування Г1–Г3: невалідний RN, використаний RN, повторний ID

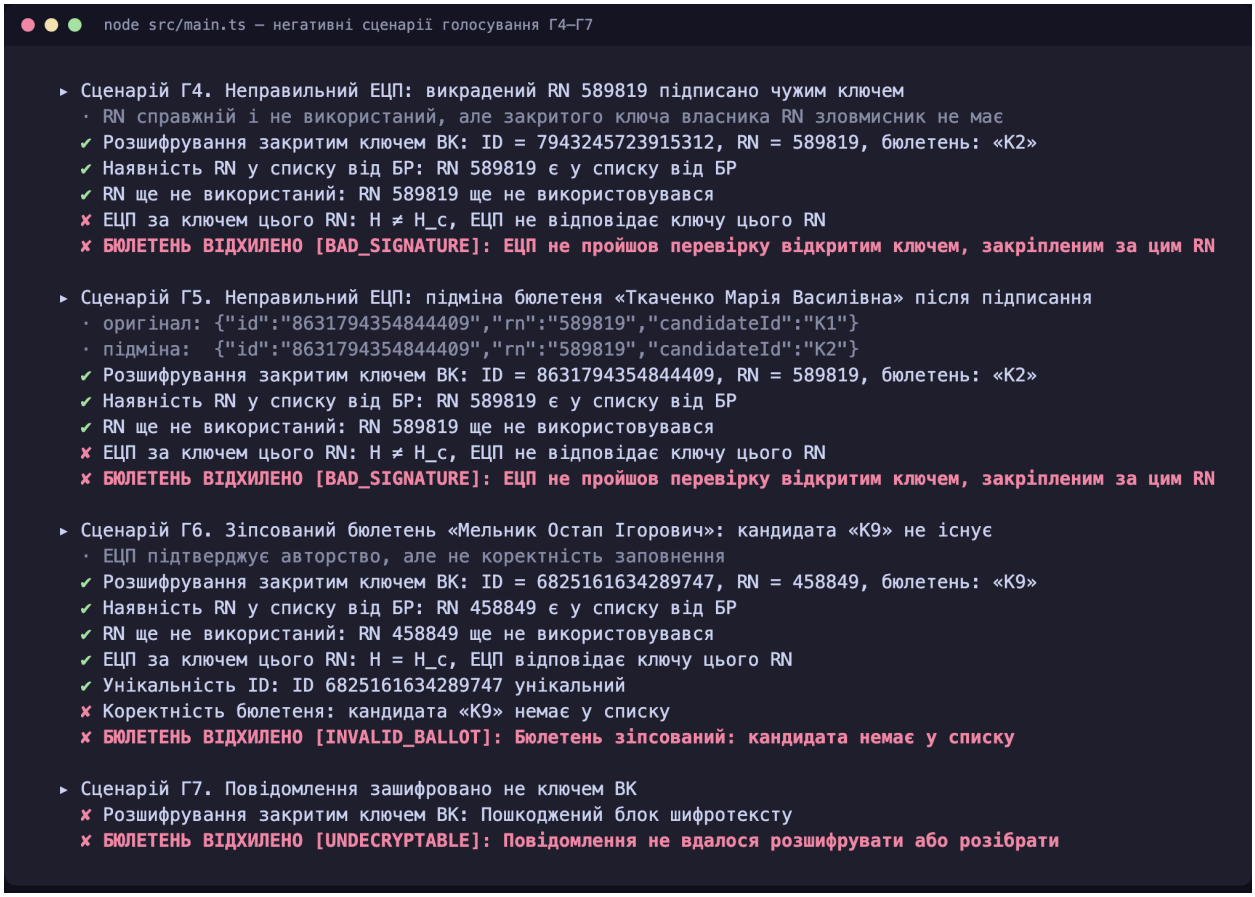


Рисунок 6 — Негативні сценарії голосування Г4–Г7: неправильний ЕЦП, зіпсований бюлетень, чужий ключ шифрування

Рисунок 7 показує, що відхилені спроби не позначили RN як використані: виборці, чиї повідомлення були підроблені чи зіпсовані, успішно голосують, після чого всі п’ять RN у реєстрі ВК позначено використаними.



Рисунок 7 — Продовження голосування та реєстр ВК після голосування

На рисунку 8 показано крок 6: підсумок 3:2 на користь першого кандидата, кількість відхилених спроб, опублікований список ID з підписаними бюлетенями та контроль за списком БР — кількість бюлетенів не перевищує кількості зареєстрованих виборців.

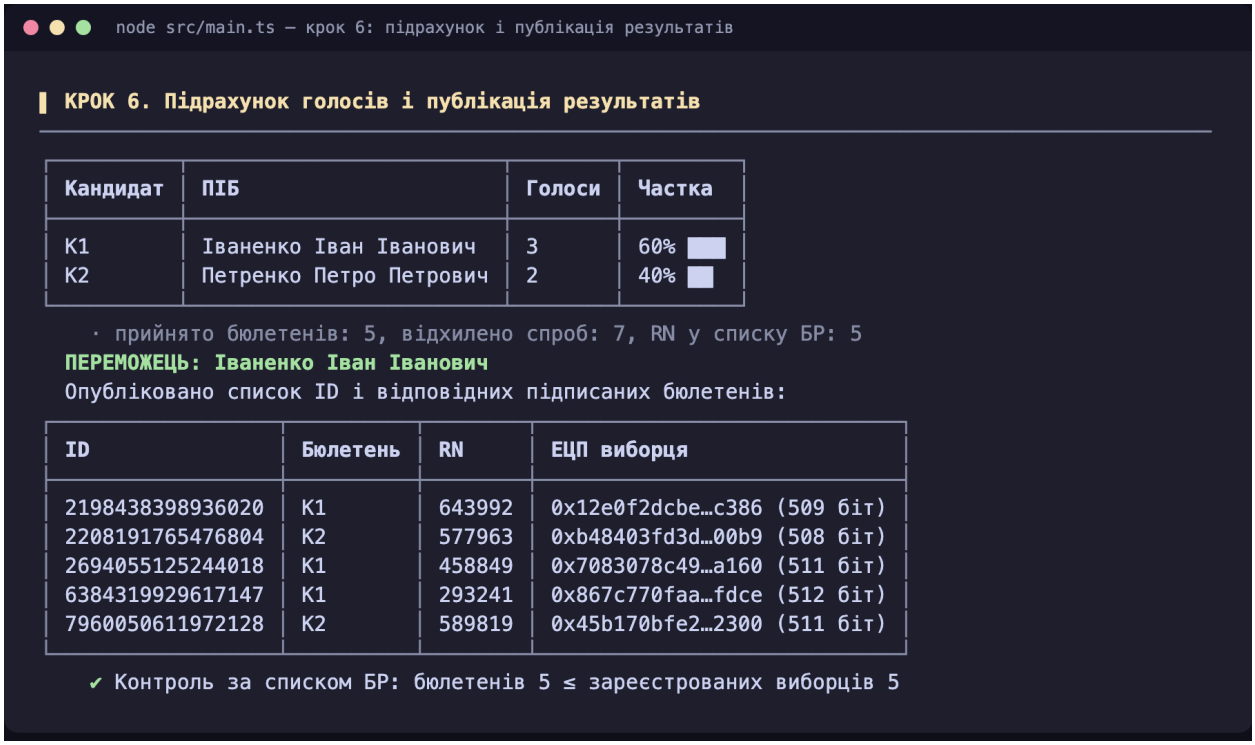


Рисунок 8 — Крок 6: підрахунок голосів і публікація результатів

Рисунок 9 демонструє перевірку виборцями власних голосів: за чесної публікації всі п’ятеро знаходять свій ID з правильним бюлетенем; у публікації, де недобросовісна ВК змінила бюлетень виборця №2 і вилучила бюлетень виборця №5, ці виборці отримують повідомлення про порушення.

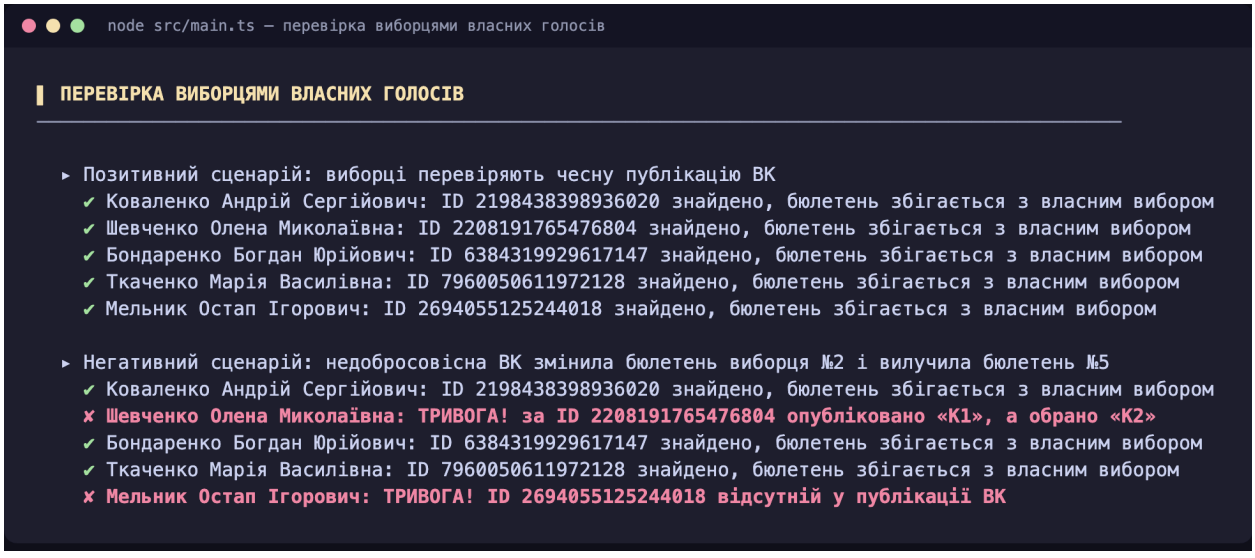


Рисунок 9 — Перевірка виборцями власних голосів (позитивний і негативний сценарії)

На рисунку 10 наведено аналіз можливостей шахрайства та підсумковий розподіл знань між сторонами. Оскільки за умовою ВК публікує підписані бюлетені, а підписаний текст містить RN, БР після публікації може

самостійно зіставити свій реєстр «ПІБ — RN» з опублікованими бюлетенями і встановити, хто як проголосував; при змові БР і ВК це можливо ще під час голосування. У протоколі з лекції 13 публікуються лише пари (Id; BL), що закриває цей канал для БР, але не для змови.



Рисунок 10 — Аналіз деанонімізації та розподіл знань між БР і ВК

4. Тестування

Окрім демонстраційного сценарію, реалізацію покрито 25 модульними тестами (команда npm test). Тести перевіряють арифметику та RSA (розширений алгоритм Евкліда, узгодженість ключів, оборотність шифрування, перевірку ЕЦП), унікальність RN, неможливість повторного отримання RN, відсутність персональних даних у списку для ВК, відхилення підробленого списку RN, а також усі позитивні й негативні сценарії перевірок ВК і перевірки виборцем власного голосу з таблиці 2.

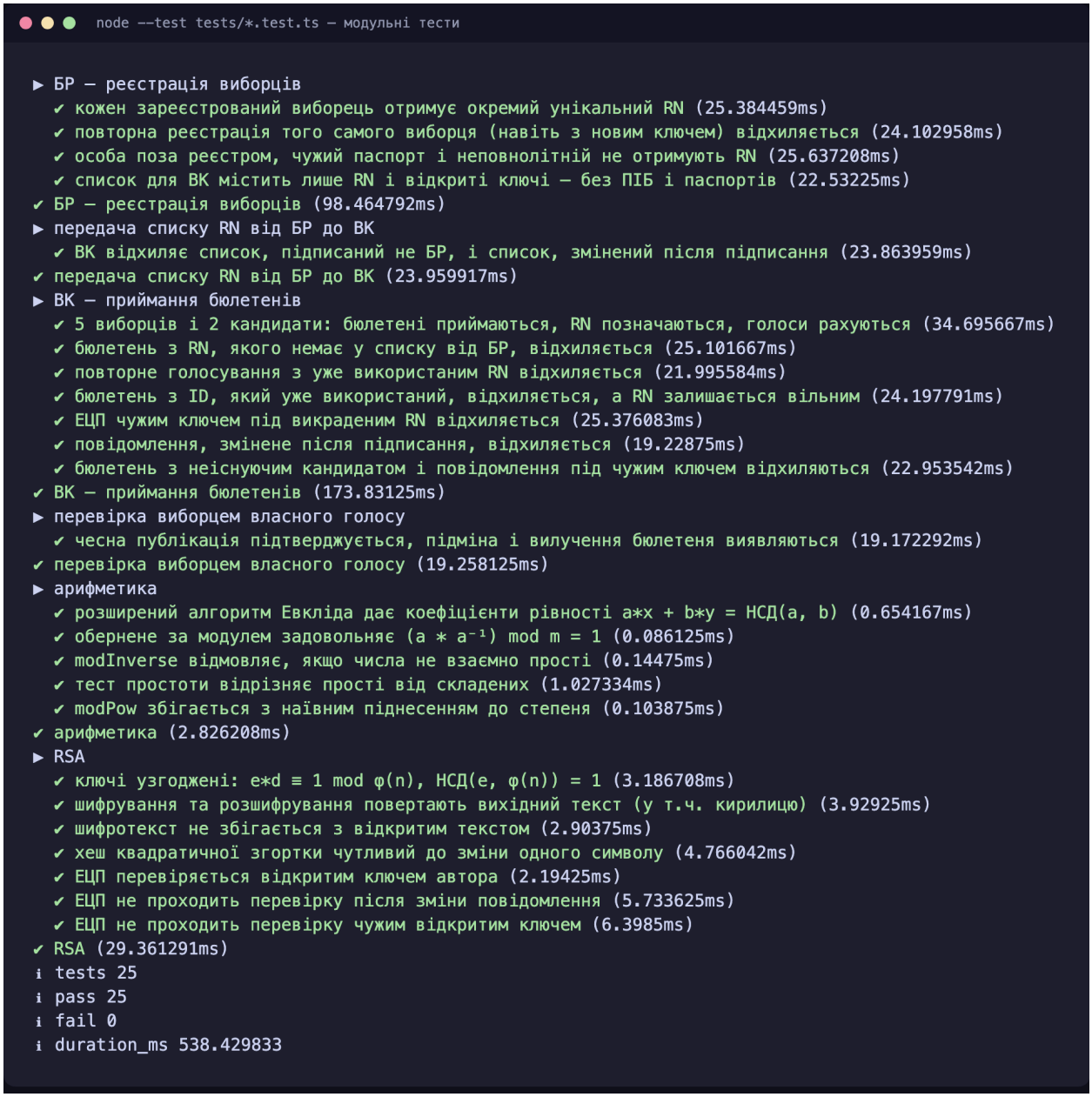


Рисунок 11 — Результати запуску модульних тестів

5. Висновок

У ході виконання лабораторної роботи програмно реалізовано всі кроки протоколу електронного голосування з розподілом повноважень між БР і ВК для двох кандидатів і п’яти виборців. БР і ВК реалізовано окремими модулями з приватними даними, що обмінюються лише підписаним списком RN без персональних даних. Реалізовано перевірки особи, права голосу та повторної реєстрації в БР, видачу унікальних випадкових RN, самостійну генерацію виборцями ключів ЕЦП і ID, підписання та шифрування повідомлень, усі перевірки ВК, підрахунок голосів, публікацію результатів і перевірку виборцем власного голосу.

Експериментально підтверджено, що протокол не допускає голосування без реєстрації (невалідний RN), повторного голосування (позначка використаного RN і заборона повторної реєстрації), голосування замість іншого та непомітної зміни бюлетеня (ЕЦП за ключем, закріпленим за RN), а також дає кожному виборцю змогу перевірити врахування свого голосу за власним ID. Порівняно з простим протоколом ЛР1 ВК більше не знає осіб виборців: у неї є лише RN, відкриті ключі та бюлетені.

Водночас розподіл повноважень лише зменшує обсяг інформації в кожної сторони, але не усуває довіри до них. ВК може використати RN виборців, які не проголосували, а БР — зареєструвати фіктивних виборців чи видати одному виборцю декілька RN; публікація БР списку виборців виявляє лише перевищення кількості бюлетенів над кількістю виборців. Публікація підписаних бюлетенів з RN дає БР змогу самостійно деанонімізувати голоси, а змова БР і ВК повністю руйнує таємницю голосування. Для усунення цих вад потрібні анонімний розподіл RN (наприклад, протокол ANDOS) або сліпий підпис.

6. Відповіді на контрольні запитання

1. Опишіть послідовність роботи протоколу електронного голосування з розподілом повноважень між БР та ВК. Виборець генерує власну пару ключів RSA для ЕЦП і звертається до БР, яка перевіряє його особу, право голосу та відсутність попередньої реєстрації. БР генерує для виборця випадковий унікальний RN і зберігає зв’язок «виборець — RN — відкритий ключ». БР передає до ВК список дійсних RN з відкритими ключами без персональних даних; ВК генерує власну пару ключів і публікує відкритий ключ. Виборець генерує випадковий ID, формує повідомлення (ID; RN; бюлетень), підписує його своїм закритим ключем, шифрує разом з ЕЦП відкритим ключем ВК і надсилає до ВК. ВК розшифровує повідомлення, перевіряє наявність і невикористаність RN, ЕЦП за ключем цього RN та унікальність ID; у разі успіху приймає бюлетень, позначає RN використаним і зберігає бюлетень з ID. Наприкінці ВК публікує результати та список ID з підписаними бюлетенями, а кожен виборець перевіряє свій голос.

2. Для чого у протоколі використовується реєстраційний номер RN та як за його допомогою контролюється повторне голосування? RN — це

знеособлене підтвердження права голосу: БР видає його лише після перевірки виборця, а ВК приймає бюлетені лише з RN зі свого списку, не знаючи, кому RN належить. Кожен RN може бути використаний для одного прийнятого бюлетеня: після прийняття ВК позначає його використаним, і наступні бюлетені з тим самим RN відхиляються. Повторне отримання RN унеможливлює БР, яка видає лише один RN на особу. Випадкова генерація RN у широкому діапазоні ускладнює його вгадування сторонньою особою.

3. Які дані зберігає БР, які дані отримує ВК та яким чином їх розділення забезпечує анонімність виборця? БР зберігає персональні дані виборця (ПІБ, паспорт), виданий йому RN і відкритий ключ ЕЦП, але не отримує бюлетенів і не знає ID. ВК отримує лише список RN з відкритими ключами і повідомлення (ID; RN; бюлетень) без імені відправника. Отже, БР знає «хто — RN», а ВК — «RN — як проголосував»; жодна сторона окремо не має зв'язку між особою та голосом, а для публікації голос пов'язується з ID, який виборець згенерував сам.

4. Для чого у протоколі використовується ЕЦП виборця та чому відкритий ключ передається до ВК разом із відповідним RN без персональних даних виборця? ЕЦП підтверджує, що повідомлення сформував саме власник RN, і що ID, RN та бюлетень не змінювалися: викрадений RN без закритого ключа власника або змінений текст повідомлення не пройдуть перевірку H = H_c. Відкритий ключ передається разом з RN, бо саме цим ключем ВК має перевіряти підпис під бюлетенем з цим RN; персональні дані ВК для цього не потрібні, а їх передача дозволила б ВК пов'язати голос з особою.

5. Як виборець може перевірити правильність врахування свого голосу після завершення голосування? Виборець зберігає згенерований ним ID і зроблений вибір. Після публікації він знаходить у списку ВК свій ID і порівнює опублікований підписаний бюлетень зі своїм. Якщо ID відсутній або бюлетень відрізняється, виборець виявляє порушення і має підняти тривогу; ВК не може непомітно змінити бюлетень, бо кожен виборець перевіряє власний запис.

6. Які можливості для шахрайства залишаються у БР та ВК окремо і які додаткові можливості виникають у разі їх змови? ВК може додати сфальсифіковані бюлетені, використовуючи невикористані RN виборців, які

не проголосували, — не перевищуючи кількості зареєстрованих, тому публікація списку виборців цього не виявить. БР може створити фіктивного виборця з власною парою ключів і RN або видати реальному виборцю декілька RN; крім того, у реалізованому варіанті БР за RN в опублікованих підписаних бюлетенях може встановити, хто як проголосував. У разі змови БР і ВК об’єднують дані «особа — RN» і «RN — бюлетень» і повністю порушують анонімність ще під час голосування, а також можуть додати фіктивних виборців до опублікованого списку і голосувати від їх імені, тож контроль за списком виборців стає марним.